Willkommen in World 1: The Terminal

JavaScript – Der Funke des Lebens für statische Webseiten.









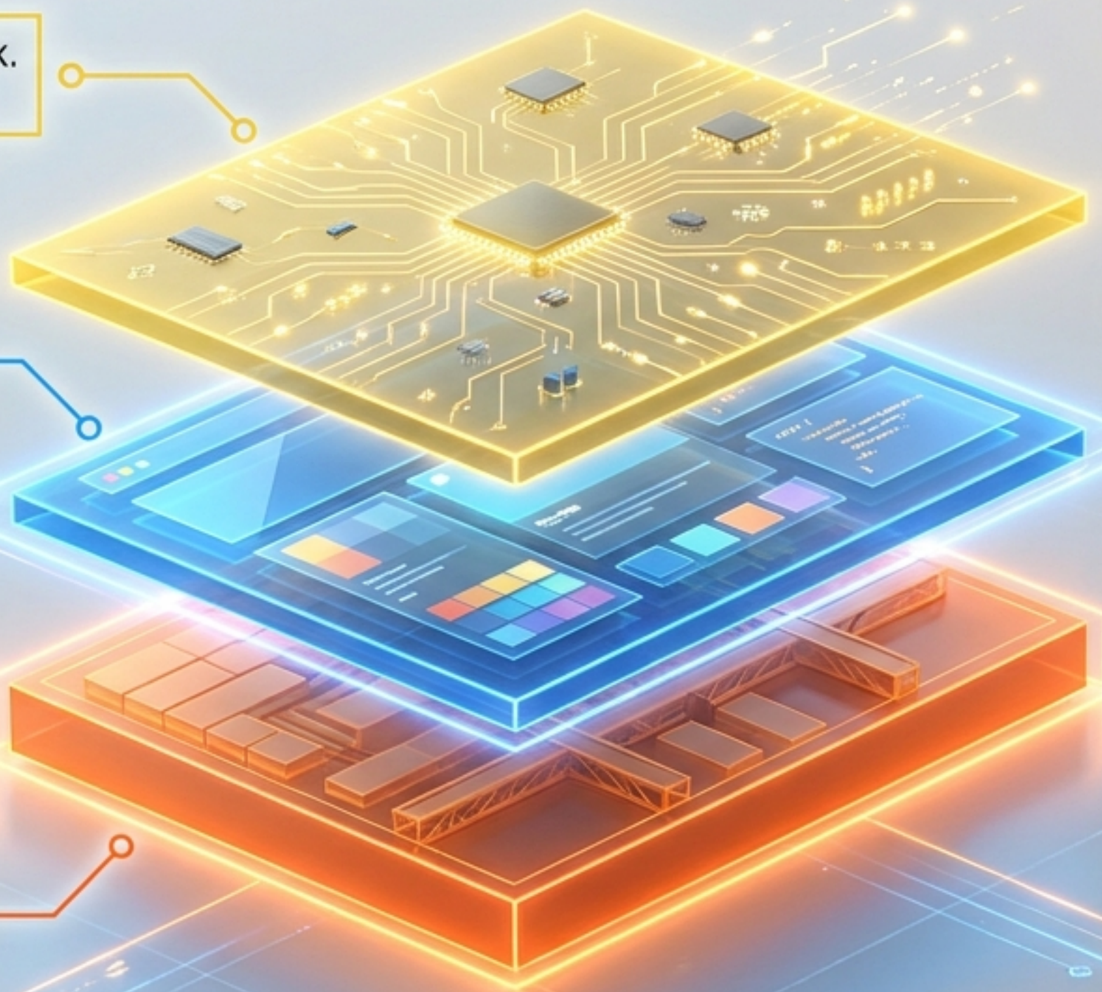
Gemini Notebook

Das Web-Dreigestirn

JavaScript – Verhalten & Dynamik.
Reagiert auf den User.

CSS – Design & Layout.
Das Aussehen der Seite.

HTML – Struktur & Inhalt.
Das Fundament der Seite.



Gemini Notebook

Der Browser ist deine Bühne (Clientseitig)



Der Server (Backend)
liefert nur die Zutaten aus.



Der Browser (Client).
Hier läuft JavaScript direkt auf deinem Gerät.
Es reagiert sofort – ohne auf die Küche warten zu müssen!

JavaScript (im Browser) kann nicht auf Dateien des Servers zugreifen – es steuert das Erlebnis am Display des Users.

Gemini Notebook

Das Portal zwischen Struktur und Verhalten

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>JS Quest - Terminal</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Meine erste JS-Quest</h1>

    <!-- Hier wird das JS-Portal geöffnet -->
    <script src="script.js"></script>
</body>
</html>
```

Binde dein Skript idealerweise am Ende des **<body>** ein, damit die HTML-Struktur zuerst geladen wird.

Gemini Notebook

Drei Wege, um mit der Welt zu sprechen

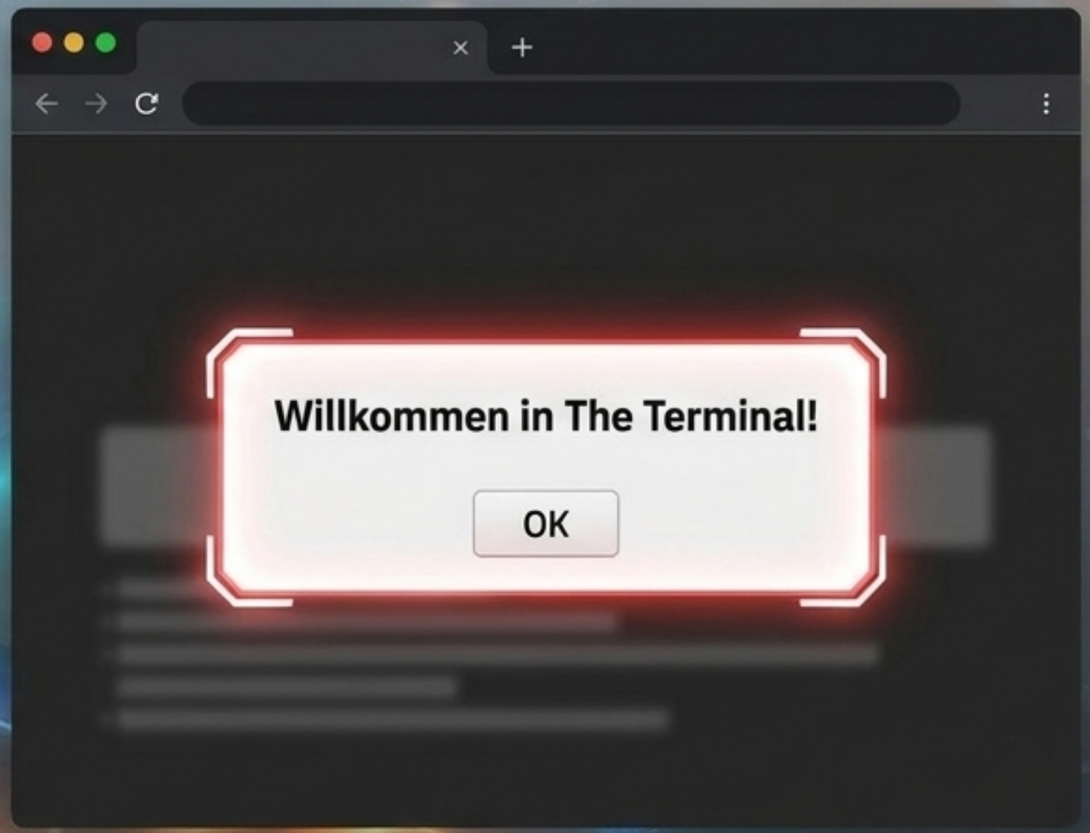
Methode	Wo passiert es?	Stört es den User?	Wofür nutzt man es?
`alert()`	Pop-up Fenster	❌ Ja - blockiert alles	Wichtige Warnungen
`console.log()`	Entwicklertools im Hintergrund	✅ Nein - unsichtbar für User	Fehleranalyse & Debugging
`innerHTML`	Direkt auf der Webseite	✅ Nein - fließendes Erlebnis	Der normale Weg für dynamische Inhalte

Gemini Notebook

Der laute Alarm: `alert()`

script.js

```
// Stoppt die komplette Ausführung des Browsers
window.alert("Willkommen in The Terminal!");
```



Das **window**-Objekt ist global. Man kann das Wort 'window' auch weglassen und einfach **`alert(...)`** schreiben. Nutze es sparsam!

Gemini Notebook

Das geheime Flüstern: `console.log()`

script.js

```
// Die Nachricht für den Entwickler
let mission = "Quest gestartet";
console.log(mission);
console.log(5 + 6);
```



Dein bester Freund bei der Fehlersuche (Debugging). Der normale User sieht diese Ausgabe nicht – du musst F12 (DevTools) drücken!

Gemini Notebook

Der Königsweg: innerHTML

index.html

```
<p id="demo">Alter Text...</p>
```

script.js

```
// Das Element suchen und den Inhalt überschreiben
document.getElementByIdId("demo").innerHTML = "Hallo World!";
```

Wir suchen eine eindeutige ID im HTML (getElementByid) und ändern deren inneren HTML-Inhalt (innerHTML). Das ist die Basis jeder dynamischen Webseite!

Gemini Notebook

Den User ins Spiel holen: prompt()

script.js

```
// Ein Eingabefeld anzeigen und die Antwort speichern
let spielerName = prompt("Wie lautet dein Name, Held?");
```



Mit `prompt()` öffnen wir einen Dialog. Die Eingabe des Users fangen wir in einem Behälter (einer Variablen) auf, um sie später zu verwenden.

Gemini Notebook

Der Informationsfluss

EINGABE
(Input)



`prompt()`

VERARBEITUNG
(Speicher)



`let name = ..`

AUSGABE
(Output)



`innerHTML`

Jedes interaktive Programm folgt diesem Kreislauf: Daten einsammeln, speichern & verarbeiten, und dem User das Ergebnis präsentieren.

Gemini Notebook

Synthese: Dein erstes interaktives Programm

index.html

```
<!DOCTYPE html>
<html>
 <body>
    <h1 id="begruessung">Warte auf Spieler...</h1>

    <script>
       // 1. Eingabe: Namen erfragen
       let held = prompt("Gib deinen Namen ein:");

       // 2. Heimliche Diagnose (optional)
       console.log("Eingeloggt als: " + held);

       // 3. Ausgabe: Die Webseite verändern
       document.getElementById("begruessung").innerHTML = "Willkommen, " + held + "!";
    </script>
 </body>
</html>
```

Gemini Notebook

Bereit für The Terminal



First Code

JS Quest: Öffne VS Code. Absolviere die Quests in World 1.

W3 Schools

Try it Yourself

W3Schools: Teste die Theorie in den Editoren (JS Output / JS Variables).



KI als Partner: Nutze KI als Denkpartner, nicht als Code-Lieferant.

("Erkläre mir den Unterschied zwischen innerHTML und alert").

Dein Abenteuer beginnt jetzt. Viel Erfolg!

Gemini Notebook